CNN Architectures

BSAD 8310: Business Forecasting — Lecture 8

Department of Economics
University of Nebraska at Omaha

August 24, 2026

Lecture Outline

Convolution: The Core Idea

Instead of connecting every neuron to every input, a convolutional filter scans locally and shares weights.

Why Convolution?

Problem with fully connected layers:

- For a 224×224 image, the input has 50,176 pixels. A single hidden layer with 1,000 neurons needs $50{,}176 \times 1{,}000 = 50\,\text{M}$ weights — just for one layer.
- Ignores spatial structure: nearby pixels are related; distant pixels usually are not.
- Does not generalize: a pattern learned in the top-left corner is not reused elsewhere.

Convolutional layer solution:

- **Local connectivity:** each neuron sees only a small **receptive field** (e.g., 3×3 pixels)
- **Weight sharing:** the same filter is applied everywhere in the image
- **Result:** far fewer parameters; translation invariance

Analogy: Scanning a document

A human reader does not memorize a font separately for each position on the page. They recognize the letter "A" once and apply it everywhere. Convolutional filters do exactly this.

How a Convolutional Layer Works

1D convolution (for sequences):

$$(f * g)[t] = \sum_{k=0}^{K-1} f[k] \cdot g[t-k]$$

Slide a filter of length K across the sequence, computing a dot product at each position.

Key parameters:

Kernel size	Length K. Larger = wider receptive field.
Stride	Steps between positions. Stride 2 halves output.
Padding	Zeros at edges (`padding='same'`).
Filters	Number of filters to learn (output channels).

What does a filter detect?

Each filter learns to detect a specific **local pattern** (e.g., edges, spikes, seasonal dips). The output is a **feature map**: high values indicate where the pattern was found.

After convolution, apply ReLU then **pooling** to reduce feature map size.

Pooling and Dropout

Pooling reduces spatial size and adds robustness. Dropout prevents co-adaptation of filters.

Pooling: Downsampling Feature Maps

After a convolutional layer, apply **pooling** to reduce the size of each feature map:

- **Max pooling:** take the maximum value in each non-overlapping window. Retains the most prominent activation — good for detecting if a feature is *present anywhere* in the window.
- **Average pooling:** take the mean. Smoother, preserves intensity information.
- **Global average pooling:** collapse the entire feature map to a single number. Commonly used before the final fully connected layer.

Pooling in PyTorch

```
nn.MaxPool1d(kernel_size=2)
nn.AvgPool1d(kernel_size=2)
nn.AdaptiveAvgPool1d(1)    (global)
```

Pooling provides **translation invariance**: a pattern detected slightly earlier or later in a sequence still produces the same pooled output.

CNN Architecture Evolution

LeNet $\rightarrow$ VGG $\rightarrow$ Inception $\rightarrow$ ResNet: each generation solved a specific limitation.

These architectures were developed for images, but the same design principles — stacking small filters, skip connections, efficient depth — carry directly into 1D CNNs for time series. Understanding the history helps you make better architectural choices.

LeNet to VGG: Depth and Simplicity

LeNet-5 (LeCun et al. 1998):

- First practical CNN (1998); ∼60K parameters
- 2 conv + 3 FC layers; 32×32 grayscale input
- Proved learned filters beat hand-crafted features

VGG-16/19 (Simonyan and Zisserman 2015):

- 16–19 layers, all 3×3 kernels
- Stack multiple 3×3 filters instead of one large filter: same receptive field, fewer parameters
- 138M parameters; strong transfer learning baseline

The 3×3 rule: Two 3×3 layers $=$ same field as one 5×5, but $2 \times 9 = 18$ vs. 25 params per channel. Three 3×3 layers match 7×7 at 27 vs. 49 params.
Stack small filters, go deep.

Inception and ResNet: Efficient Depth

GoogLeNet / Inception (Szegedy et al. 2015):

- Inception module: 1×1, 3×3, 5×5 filters **in parallel**, outputs concatenated
- 1×1 convolutions reduce channel depth (bottleneck)
- 22 layers; only 5M params vs. VGG's 138M

ResNet (He et al. 2016):

- **Residual connections:** $\mathbf{a}^{(\ell+2)} = \mathcal{F}(\mathbf{a}^{(\ell)}) + \mathbf{a}^{(\ell)}$
- Gradient flows through the skip connection — solves vanishing gradients in deep nets
- Enables 50, 101, 152-layer stable training

Residual (Skip) Connection

$$\mathbf{a}^{(\ell+2)} = \underbrace{\mathcal{F}(\mathbf{a}^{(\ell)})}_{\text{residual}} + \underbrace{\mathbf{a}^{(\ell)}}_{\text{identity}}$$

If the layer learns nothing useful, set $\mathcal{F} \approx 0$ and pass input unchanged. The network learns only the *residual*.

Skip connections are now standard: Transformers, U-Nets, and most modern architectures include them.

1D CNNs for Time Series

Apply convolutional filters along the time axis to detect local temporal patterns automatically.

Why Use a 1D CNN for Forecasting?

A **1D CNN** applies filters along the *time* dimension:

- Each filter detects a local temporal pattern (spike, dip, ramp over K steps)
- Multiple parallel filters learn different patterns
- Stacking layers widens the look-back: 2 layers of kernel size 5 see 9 steps total

vs. lag features + XGBoost:

- No manual feature engineering needed
- Patterns are learned, not hand-coded
- Tradeoff: needs more data to learn effectively

1D CNN in PyTorch

```
nn.Conv1d(
in_channels=1,
out_channels=32,
kernel_size=5,
padding=2
)
```

Input: `(batch, channels, seq_len)`

Univariate: `in_channels=1`; multivariate: `in_channels=p`

1D CNNs are fast to train and competitive with LSTMs on shorter sequences ($<$500 steps).

1D CNN Forecasting Pipeline in PyTorch

CNN Regressor for Time Series

```
class CNNForecaster(nn.Module):

def __init__(self, seq_len, n_feat):

self.conv = nn.Sequential(

nn.Conv1d(n_feat, 32, kernel_size=5, padding=2),

nn.ReLU(),

nn.Conv1d(32, 64, kernel_size=3, padding=1),

nn.ReLU(),

nn.AdaptiveAvgPool1d(1)

)

self.fc = nn.Linear(64, 1)

def forward(self, x):

return self.fc(self.conv(x).squeeze(-1)).squeeze(-1)
```

Architecture walkthrough:

1. `Conv1d(n_feat, 32, 5)`: 32 filters, width 5
2. `ReLU`: non-linearity
3. `Conv1d(32, 64, 3)`: 64 filters, width 3
4. `AdaptiveAvgPool1d(1)`: global avg pool
5. `Linear(64, 1)`: forecast output

Typical RSXFS result: 1D CNN RMSE $\approx$ 1,600–1,800.

Key Takeaways

CNN architectures: from local filters to deep residual networks.

CNN Architecture Summary

Architecture	Year	Key Innovation	Params
LeNet	1998	First practical CNN; conv + pool	∼60K
VGG	2015	Deep stack of 3×3 filters	∼138M
Inception	2015	Parallel multi-scale filters; 1×1 bottleneck	∼5M
ResNet	2016	Residual (skip) connections; train 100+ layers	∼25M

For business forecasting with 1D CNNs:

- Use 2–3 `Conv1d` layers with kernel size 3–7
- Follow with ReLU and global average pooling
- Skip connections help if stacking more than 4 layers
- Faster to train than LSTMs; competitive accuracy on shorter sequences

Lecture 8: Key Takeaways

1. **Convolutional layers** apply learned filters locally across input positions. This gives **weight sharing** (same filter reused everywhere) and **translation invariance** (pattern recognized regardless of position).
2. **Pooling** downsamples feature maps. Max pooling retains peak activations; global average pooling collapses each feature map to a single value before the output layer.
3. The 3×3 rule: stack small filters (VGG principle). Two 3×3 layers cover the same field as one 5×5 with fewer parameters.
4. **Residual connections** (ResNet) add the input directly to the layer output: $\mathbf{a}^{(\ell+2)} = \mathcal{F}(\mathbf{a}^{(\ell)}) + \mathbf{a}^{(\ell)}$. This stabilizes gradients in very deep networks.
5. **1D CNNs** apply filters along the time axis to detect local temporal patterns. Faster to train than LSTMs; a strong alternative for shorter sequences.

Next: RNNs, LSTMs & Transformers — architectures with explicit memory and attention over sequences.

References I

- He, Kaiming et al. (2016). "Deep Residual Learning for Image Recognition". In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.
- LeCun, Yann et al. (1998). "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11, pp. 2278–2324.
- Simonyan, Karen and Andrew Zisserman (2015). "Very Deep Convolutional Networks for Large-Scale Image Recognition". In: *International Conference on Learning Representations (ICLR)*.
- Szegedy, Christian et al. (2015). "Going Deeper with Convolutions". In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1–9.